Thus, the total operations count yields $O(n \cdot \log n + k\sqrt{n})$, on the average. This should be contrasted with $O(k \cdot n)$ for the straightforward algorithm.

It is instructive to compare the above with the situation when the Euclidean norm is used. There we need to compute

$$\|q^{(s)} - p\|_2^2 = \|q^{(s)}\|_2^2 + \|p\|_2^2 - 2\sum_{i=1}^{n} p_i q_i^{(s)}, \qquad s = 1, 2, \cdots, k.$$

Assuming that $\|q^{(s)}\|_2^2$, $s = 1, 2, \cdots, k$ can be computed as part of the preconditioning process, and disregarding the evaluation of $\|p\|^2$, we see that the above computation requires the matrix $*$ vector product $Q * p$ where $\{Q\}_{ij} = q_i^{(j)}$. It has been shown by Winograd [2] that this product requires at least $0.5k \cdot n$ multiplications, even when preconditioning on Q is allowed.

Application 2: A problem in computational geometry.

In [3], Dobkin and Lipton posed the problem of efficient evaluation of several geometrical queries, one of them being the following. "Given n straight lines in the plane and a point (x_0, y_0), determine whether this point lies on one of the lines." The assumption is that preprocessing on the n lines is free and the criteria for the efficiency of the solution are the time T it takes to answer the query and the space S it takes to store the preprocessed data.

In the above mentioned paper, the authors provide a geometrical solution where $T = O(\log n)$ and $S = O(n^3)$. Several subsequent papers (e.g., [4], [5]), although addressing a more general problem, provide more efficient solutions; for instance, one can deduce from the results of [5] an algorithm with $T = O(\log n)$, $S = O(n^2)$.

Here, we present an algorithm which runs in minimum space ($S = O(n)$) and has average running time $O(\sqrt{n})$. Assume that $y = a_i x + b_i$ $i = 1, 2, \cdots, n$ are the equations of the n lines. We consider the problem of evaluating $d = \min|(a_i x_0 + b_i) - y_0|$. Corollary 1 lends itself to this case in the following way. Preprocess $a = (a_1, a_2, \cdots, a_n)$, $b = (b_1, b_2, \cdots b_n)$ as in Theorem 1, steps P1–P3.

Define the two sequences

$$a' = (a_1^* x_0, a_2^* x_0, \cdots, a_n^* x_0),$$

$$b = (b_1 - y_0, b_2 - y_0, \cdots, b_n - y_0).$$

Then a', b' can be considered to be preconditioned as well, for the permutations i and k found for a and b in Steps P1–P3 are good for a' and b' (if $x_0 < 0$, permutation i has to be reversed).

Therefore, for a given (x_0, y_0) it takes $T = O(\sqrt{n})$ steps on the average to resolve the query. Note that in fact we achieve more than just a "yes or no" answer to the query; we find the minimum of the y-distances between (x_0, y_0) and the given lines.

As an added bonus, note that a very similar algorithm can be used for the dual query, namely, "given a fixed set of n points in the plane and a straight line, determine if the line passes through any of the points."

REFERENCES

[1] R. O. Duda and P. E. Hart, *Pattern Classification and Scene Analysis*. New York: Wiley, 1973, p. 98.

[2] S. Winograd, "On the number of multiplications necessary to compute certain functions," *Comm. Pure Appl. Math.*, vol. 24, pp. 165–179, 1970.

[3] D. P. Dobkin and R. J. Lipton, "Multidimensional searching problems," *SIAM J. Comput.*, vol. 5, pp. 181–186, 1976.

[4] D. T. Lee and F. P. Preparata, "Location of a point in a planar subdivision and its applications," *SIAM J. Comput.*, vol. 6, Sept. 1977.

[5] R. J. Lipton and R. E. Tarjan, "Applications of a planar separator theorem," in *Proc. 18th Symp. Foundations Comput. Sci.*, Providence, RI, Oct. 1977, pp. 162–170.

Pseudorandom Arrays for Built-In Tests

P. H. BARDELL, AND W. H. MCANNEY

Abstract **— Parallel pseudorandom sequences for use in built-in test are discussed. The two-dimensional nature of these sequences makes it natural to consider the resulting binary arrays. Some of the desired properties of such arrays are discussed, as well as some of the problems. Generators for such arrays are described. A conventional LFSR with parallel output is shown to be a poor choice for such a generator. Several compact generators are described, which are shown to be compromises between complexity and varying degrees of implementation of the desired properties in the resulting sequences. One of the compact generators produces sequences which have the desired properties for built-in tests.**

Index Terms **— Built-in self-test stimuli, linear dependencies in *m*-sequences, parallel LFSR sequences, pseudorandom binary sequences, pseudorandom sequences, two-dimensional window property.**

INTRODUCTION

With the advent of VLSI and the ever-increasing size of digital networks, the problems of test generation have caused some workers to consider using some of the available circuitry for implementing a portion of the test function within the network. This has led to the increasing interest in self-test or built-in test. Simply put, a built-in test puts a stimulus source (pattern generator) and a response evaluator (compressor) into the network to perform all or part of the test [5], [12].

One of the key elements of many of these proposals is the pattern generator. Pseudorandom patterns are one type of stimuli that has received a lot of interest. This is primarily due to the fact that they can be derived algorithmically with a very simple structure, namely, a linear feedback shift register (LFSR). Considerable literature exists on the properties of LFSR's [9] and we will draw from that in later sections.

Many of these built-in test schemes utilize the scan-path features of existing design-for-testability disciplines (used to reduce the testing problem to a tractable combinational network problem). The common complaint of long test application times due to loading long scan strings is addressed by a parallel approach. The scan strings are broken into several shorter strings to speed the test [1], [8]. In these structures, it is necessary to feed several scan paths with test stimuli in parallel. If the test stimuli are to be pseudorandom sequences, the characteristics of parallel sequence generators must be looked at. The problem we address is to feed several scan paths in parallel so that the resulting array of pseudorandom patterns is a satisfactory stimulus for the digital network that is contained within the scan path.

By a "satisfactory stimulus" we mean that no conditions are imposed by the structure or the sequence generator such that the inputs to the logic under test are precluded, *a priori*, from observing all possible binary patterns. This could be called the "philosophy of possible exhaustion," in that as a k-input subnetwork is driven by a generator of this type, eventually all 2^k binary k-tuples will be presented to the network inputs. While in practice one may choose not to wait until this event is assured, this condition does not exclude the possibility at the start.

Another condition that is desirable is that no special conditions be imposed on the logic or the assignment of shift register latches (SRL's) to specific logic inputs (no special encoding of the inputs). In particular, any circuit can be fed by any combination of the scan latches so a two dimensional array of pseudorandom bits is required.

Manuscript received June 8, 1984; revised November 14, 1984.
The authors are with IBM, Poughkeepsie, NY 12602.
IEEE Log Number 8609148.

0018-9340/86/0700-0653$01.00 © 1986 IEEE

Authorized licensed use limited to: Lakehead University. Downloaded on October 01,2026 at 13:52:26 UTC from IEEE Xplore. Restrictions apply.

With this condition in mind, we will discuss several such generators in the following sections.

In the test protocol known as bit-pushing [12], the random sequence is advanced one shift and applied to the circuit under test. The parallel scan path structures for which we want to construct stimulus generators require that an array of pseudorandom bits be applied as a single stimulus and the results of that application be scanned into a response compressor such as a multiple-input signature register (MISR). This requirement that the response of the circuit to the array of stimulus bits be captured by shifting the contents of the scan path into the MISR precludes use of a bit-pushing test protocol. Another test protocol called shift and flip has been proposed by Barzilai and Rosen [4]. In this scheme, each stage of the scan path has the ability to invert the data in the latch in a pseudorandom manner. Implementation of this would require either a more complex SRL stage, or another pseudorandom generator that drives the controls of all of the scan latches involved. Due to the complexity of the added circuitry required we will not consider the shift and flip protocol in the following.

We will 1) review the properties of bit sequences obtained from linear feedback shift registers, since these are the prime candidates for our generators; 2) examine the deficiencies of a linear feedback shift register as a parallel generator; 3) define a two-dimensional window property and present a method generating parallel sequences with this property; and, 4) describe a technique to distribute a pseudorandom sequence over an arbitrary group of parallel scan paths so that no overlap occurs between any of the scan paths.

Properties of Shift-Register Sequences

In this section, we review the key properties of pseudorandom sequences generated by LFSR's. Such a shift register is shown in Fig. 1. The feedback which is summed (modulo 2) to form the next state can be described by a polynomial as in the figure.

For a shift register with n stages, the feedback will be described by an nth degree polynomial, the characteristic polynomial. The feedback contains only addition (modulo 2) which is a linear operation in $GF(2)$, hence the term linear feedback shift register. Other feedback configurations with nonlinear operators (e.g., AND, OR) have been studied by de Bruijn [7] and others. These will not be treated in this paper.

If the nth degree characteristic polynomial $p(x)$ is irreducible (cannot be factored) and if

$$\frac{x^m - 1 = 0 \bmod p(x) \quad \text{for } m = 2^n - 1}{p(x) \quad \neq 0 \bmod p(x) \quad \text{for } m < 2^n - 1}$$

$p(x)$ is said to be a primitive polynomial. If $p(x)$ is primitive, the number of states that the LFSR will cycle through is $2^n - 1$. There exists at least one primitive polynomial of degree n for every n [14]. The all "0" state is precluded as an initial state and does not appear in the $2^n - 1$ states. If the output as shown in Fig. 1 is observed as the LFSR cycles through its $2^n - 1$ states, a pseudorandom sequence $\{a_i\}$ is observed. This sequence is sometimes called a maximal length shift-register sequence or an m-sequence.

Since any one of $2^n - 1$ possible states of the LFSR could have been the initial state or "seed," there are $2^n - 1$ m-sequences that can be generated by the LFSR. We define this as the group M. This gives rise to some interesting properties.

Property 1: The shift property. Any cyclic shift of $\{a_i\}$ belonging to M is also a member of M.

Property 2: The addition property. The sum (modulo 2) of any two $\{a_i\}$ belonging to M is also a member of M.

Property 3: The shift and add property. The sum of a sequence $\{a_i\}$ belonging to M and a cyclic shift of itself is also a member of M.

Property 4: The window property. If a window of width n is slid along an m-sequence belonging to M, each of the $2^n - 1$ binary n-tuples will be seen exactly once. To avoid difficulties at the ends, imagine that the sequence is written in a circle.

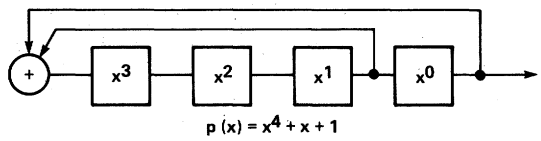


Fig. 1. Linear feedback shift register.

Any binary sequence of length m, $a = a_0 a_1 \cdots a_{m-1}$, can be represented by the polynomial

$$a(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{m-1} x^{m-1}.$$

A cyclic shift of $a(x)$ by one place to the right $a_{m-1} a_0 \cdots a_{m-2}$ is represented by

$$a_{m-1} + a_0 x + a_1 x^2 + \cdots + a_{m-2} x^{m-1}.$$

Since we are dealing with a sequence of length m, we define $x^m = 1$ (m shifts leave the sequence unchanged). Then the shifted sequence is

$$\begin{aligned} xa(x) &= a_0 x + a_1 x^2 + a_2 x^3 + \cdots + a_{m-2} x^{m-1} + a_{m-1} x^m \\ &= a_{m-1} + a_0 x + a_1 x^2 + \cdots + a_{m-2} x^{m-1}. \end{aligned}$$

Thus, multiplying by x corresponds to a cyclic shift to the right.

LFSR's as Parallel Sequence Generators

The first implementation that comes to mind is simply to take the output from each stage of an LFSR and feed it into one of the scan paths. In this scheme, the LFSR shifting clocks and the scan path shifting clocks are the same (Fig. 2). This direct feed implementation results in an array of pseudorandom bits that has a "structural dependence." If adjacent scan paths are fed from contiguous LFSR stages, the same values appear in the adjacent scan paths offset by one shift. In particular, if a 2×2 window is placed anywhere on the array formed by the scan paths as shown in Fig. 2, only half of the possible 2×2 binary arrays will appear as the LFSR is shifted through its cycle. If a 3×3 window is considered, only one fourth of the possible binary arrays appear.

Larger spans have proportionally smaller coverage. This is a serious structural dependence in the sense of our application. Consider the example shown in Fig. 3. It is desired to test the AND gate input 1, stuck-at-1. If it is driven by the scan path connections shown, the tests [0 1 1] and [1 0 1] never appear. Thus, the circuit connected in this manner sees only half of its test set. While this simple example could be remedied by a reassignment of scan latches for the inputs, we do not allow that as a solution. Recall from the discussion in the Introduction, the sequence generator must be such that reassignment is not necessary to insure adequate coverage of the pseudorandom pattern set. Thus, something more than simple LFSR parallel feed is required as a parallel sequence generator.

In addition to the class of structural dependencies shown above, the m-sequence itself has "linear dependencies" which have their origin in the fundamental recursion that generates the sequence. Consider the example in Fig. 4. The recursion that generates the sequence is

$$x^4 = x + 1.$$

When the network is connected as shown, the test [1 1 1] does not occur since the third input is always the sum (mod 2) of the first two. Here again, reassignment of the inputs to the circuit under test would avoid the problem, but we are interested in a more general solution.

Several parallel pseudorandom sequence generators have been proposed in the literature. Hsiao proposed a parallel generator [10] that on the surface would seem to have the properties we seek. However, the construction technique exploits the linear dependencies of the sequence and results in many local occurrences of the linear de-

Authorized licensed use limited to: Lakehead University. Downloaded on October 01,2026 at 13:52:26 UTC from IEEE Xplore. Restrictions apply.

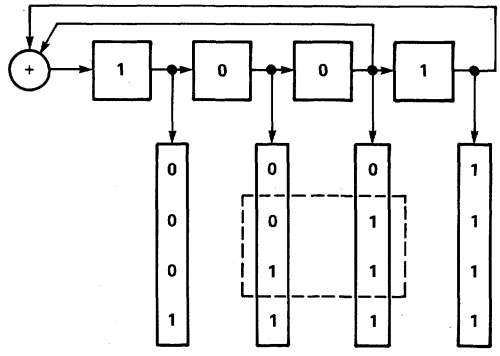


Fig. 2. Pseudorandom array from LFSR feed.

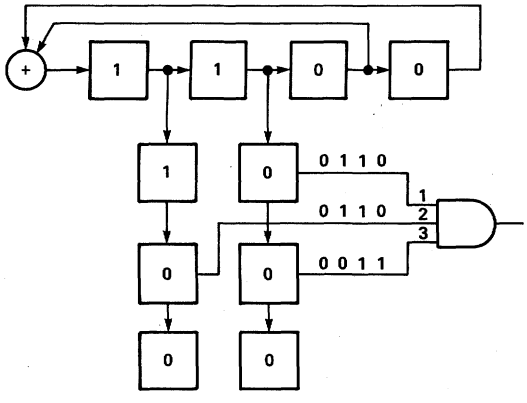


Fig. 3. Effect of structural dependence.

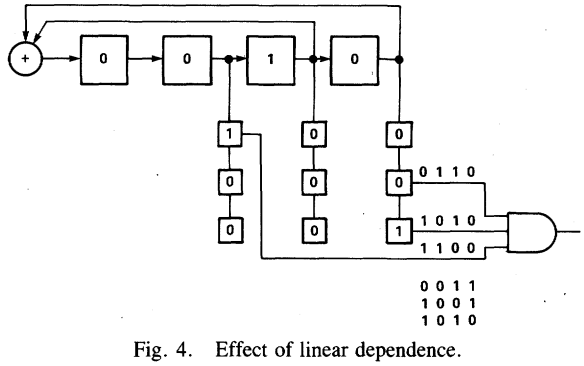


Fig. 4. Effect of linear dependence.

pendencies. Hurd [11] has proposed another generator that has several outputs that could be used as a parallel generator. This scheme also exploits the linear dependencies inherent to the m-sequence, thus concentrating the linear dependencies into the array making it an unsuitable generator for built-in-test purposes.

Two-Dimensional Window Generator

Many applications for two-dimensional pseudorandom arrays and their construction have appeared in the literature [14]–[16]. In this paper, it is proposed that the array be implemented as the states of a group of i-bit shift registers. Consider an array of k shift registers, each of i stages (Fig. 5), called the $i \times k$ array A_m. We desire to feed the m-sequence from an n-stage LFSR, where $n = rh$, into the shift register array so that the following property exists.

Two-Dimensional Window Property: If an $r \times h$ subspace (window) is defined on $A_m(i \times k)$, each of the $2^{rh} - 1$ nonzero binary $r \times h$ arrays will appear in the window exactly once in $2^{rh} - 1$ shifts of the LFSR.

This window property is different from that defined by MacWilliams and Sloane [14], but it is defined in a way that is useful with built-in test structures like STUMPS [1] which have parallel scan paths. As will be shown, parallel bit streams can be shifted into the scan paths in such a way as to form this two-dimensional window.

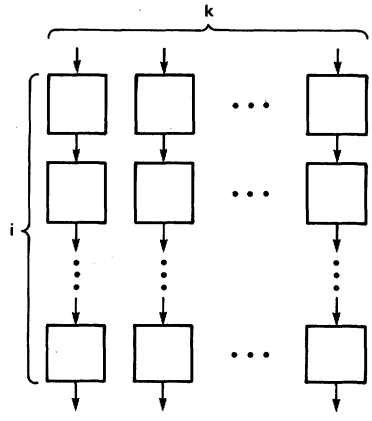


Fig. 5. $i \times k$ shift register array A_m.

To construct a pseudorandom array with this property, one chooses a tap point in the sequence, say a_j, to feed into the top of column 0 (shift register 1). A tap shifted s shifts a_{j+s} is fed into the second column, and so forth. At a given instant in time, the sequence appearing at the top of the columns

$$\{a\} = a_j, a_{j+s}, a_{j+2s}, \cdots, a_{j+(h-1)s}$$

can be expressed as a polynomial

$$f_m(x) = a_j x^0 + a_{j+s}x^s + \cdots + a_{j+(h-1)s}x^{(h-1)s}.$$

As the pattern is shifted down one stage and a new row appears at the top, the expression becomes

$$xf_m(x) = a_j x + a_{j+s}x^{s+1} + a_{j+2s}x^{2s+1} + \cdots + a_{j+(h-1)s}x^{(h-1)s+1}.$$

After $r - 1$ shifts, to fill the h columns r deep, we have

$$x^{r-1}f_m(x) = a_j x^{r-1} + a_{j+s}x^{s+r-1} + a_{j+2s}x^{2s+r-1} + \cdots + a_{j+(h-1)s}x^{(h-1)s+r-1}$$

at the bottom of the $r \times h$ window and

$$f'_m(x) = a_{j+r-1} + a_{j+r-1+s}x^s + a_{j+r-1+2s}x^{2s} + \cdots + a_{j+r-1+(h-1)s}x^{(h-1)s}$$

at the top. Fig. 6 shows the resulting array. The entire $r \times h$ window can be described by

$$\begin{aligned} f(x) = {} & a_{j+r-1} + a_{j+r-1+s}x^s + \cdots + a_{j+r-1+(h-1)s}x^{(h-1)s} \\ & + a_{j+r-2}x + a_{j+r-2+s}x^{s+1} + \cdots \\ & + a_{j+r-2+(h-1)s}x^{(h-1)s+1} \\ & \vdots \\ & + a_j x^{r-1} + a_{j+s}x^{s+r-1} + \cdots + a_{j+(h-1)s}x^{(h-1)s+r-1}. \end{aligned}$$

Thus, the $r \times h$ window we are interested in can be described by the polynomial $f(x)$ which is of degree $t = (h - 1)s + r - 1$. If we let $s = r$, t becomes

$$t = rh - 1.$$

Since $rh = n$, $t < n$. Under these conditions, we note that the $r \times h$ members of $\{a_i\}$ appear once and only once in the window. In effect we have "folded" the sequence into the array. We "pushed" the

Authorized licensed use limited to: Lakehead University. Downloaded on October 01,2026 at 13:52:26 UTC from IEEE Xplore. Restrictions apply.

m-sequence: $a = a_0 a_1 \cdots a_j a_{j+1} \cdots a_{j+s} \cdots a_{j+2s} \cdots a_{2r-2} a_0 \cdots$

a_{j+r-1}	$a_{j+r-1+s}$	$a_{j+r-1+2s}$	$\cdots$	$a_{j+r-1+(h-1)s}$
.	.	.		.
.	.	.		.
.	.	.		.
a_{j+2}	a_{j+s+2}	a_{j+2s+2}	$\cdots$	$a_{j+(h-1)s+2}$
a_{j+1}	a_{j+s+1}	a_{j+2s+1}	$\cdots$	$a_{j+(h-1)s+1}$
a_j	a_{j+s}	a_{j+2s}	$\cdots$	$a_{j+(h-1)s}$

Fig. 6. Construction of a two-dimensional pseudorandom array.

first n bits into column 0, the second n bits into the next, and so on.

Since the LFSR with primitive characteristic polynomial $p(x)$ of degree n will generate all $2^n - 1$ sequences, all $2^n - 1$ distinct $r \times h$ binary arrays will appear in the window.

Remembering that the window is a subspace of the m sequence generated by $p(x)$, and is described by $f(x)$, a polynomial of degree $t < n$, we have

$$f(x) \neq 0 \bmod p(x)$$

a necessary and sufficient condition for complete coverage of the subspace described by $f(x)$ [3].

Fig. 7 shows a simple example of a 2×2 window array driven by a 4-stage LFSR. As shown, it is not particularly interesting, but the reader can verify that indeed all 15 nonzero 2×2 binary arrays do appear in the window as the LFSR is cycled through its states. However, if instead of a 2×2 array of shift-register stages, a larger, say, 4×6 array is used, and it is still desired to have a 2×2 window at each position in the array, the construction becomes more interesting. Fig. 8(a) shows such a construction using a 10-stage idler-register to hold 10 shifted outputs of the LFSR.

It should be noted that an arbitrarily chosen window, such as that shown, contains 4 consecutive members of the m-sequence, so the window property holds. The columns of the array are driven by taps 0, 2, 4, 6, 8, and 10 of the idler register. Since the LFSR has only four stages, it is convenient to use the shift and add property of m-sequences to obtain the required inputs directly from the LFSR. We note the recursion $a_0 + a_1 + a_4 = 0$.

Therefore,

$$a_4 = a_0 + a_1$$

$$a_6 = a_2 + a_3$$

$$a_8 = a_4 + a_5 = a_0 + a_1 + a_1 + a_2 = a_0 + a_2$$

$$a_{10} = a_6 + a_7 = a_2 + a_3 + a_3 + a_4 = a_2 + a_0 + a_1.$$

This implementation is shown in Fig. 8(b). The above construction is similar to that of Tsao [18]. Latawiec [13] has also shown a procedure that can be adapted to this construction. The entries in the array appear in reverse order from those in the array in Fig. 8(a), because the stages of the LFSR are ordered from right to left while the stages of the idler register are ordered from left to right. This construction can be extended to any number of columns (of any depth). After $2^n - 1$ (or 15 in our example), the columns merely repeat. A 2×2 window placed anywhere on this plane will observe all $2^4 - 1$ nonzero binary patterns once and only once in $2^4 - 1$ consecutive shifts of the LFSR.

Thus, we have shown the construction of a pseudorandom array with the two-dimensional window property. One apparent deficiency in the construction just shown is the absence of the all-zero n-tuple from the window. This property of m-sequences is avoided by some workers by using a nonlinear feedback network to achieve the all-zero state in the shift register. These de Bruijn sequences [7] do not obey the shift-and-add property that is required to construct the window property outlined above. A simple alternative is to make the LFSR in the generator one stage longer. In the example shown, the all-zero 2×2 array would occur once every 31 shifts, while the other patterns would occur once each 15 shifts.

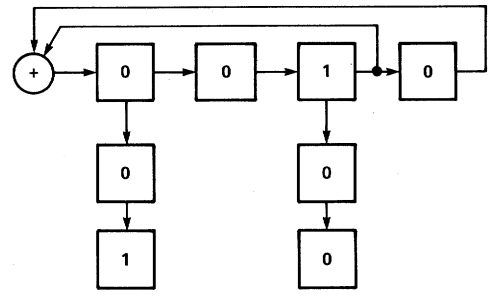


Fig. 7. Two-dimensional (2×2) window.

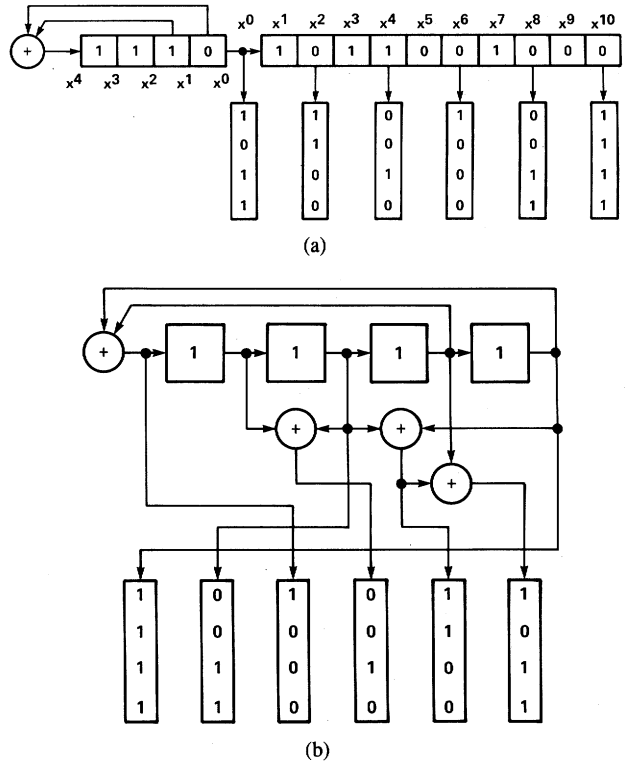


Fig. 8. (a) A 4×6 array with a 2×2 window. (b) Compact construction of 4×6 array with a 2×2 window.

A corollary property of the array construction shown above follows: if the array A_m is $w \times w$ in extent where $w = 2^n - 1$, an $r \times h$ window slid vertically over A_m in a cyclic manner (consider A_m a torus) will expose each of the $2^{rh} - 1$ nonzero $r \times h$ binary arrays exactly once. The property holds for horizontal shifts also, but not for combinations.

Another way to implement a two-dimensional $r \times h$ window would be to have h distinct LFSR's with the same characteristic polynomial of degree at least rh. One could drive each of the h columns of the window with one of these generators. The window property would be obtained by seeding LFSR_i with a state that is r shifts advanced over that seeded in LFSR_{i-1}. As the h LFSR's cycle through their sequences, the h shift-register channels (each of length r) will display the window property. Thus, if we want to have an array of h shift registers each r stages long to exhibit the two-dimensional window property, we have our choice of implementations, one LFSR of length rh or greater with its exclusive-OR network, or h of them, each rh stages long feeding the scan paths directly. Cost of implementation would dictate the single LFSR in most cases.

In this section, we have been concerned with constructing a generator that will exhaust a k subspace within a $r \times h$ window where $k < rh$. We showed that an LFSR with rh stages is required. If the

Authorized licensed use limited to: Lakehead University. Downloaded on October 01,2026 at 13:52:26 UTC from IEEE Xplore. Restrictions apply.

window is required to approach the size of the entire scan path structure or the $i \times k$ array A_m, a large number of stages are required in the LFSR. In the limit, one stage in the LFSR would be required for each stage in the scan path. For a network with 500 scan latches, 500 stages would be required in the LFSR! Tang and Chen [17] have proposed a test generation technique using LFSR's which is based on linear codes. In this case, one would also need a large degree LFSR to implement a test set that would exhaust a k-subspace in a large structure. In particular, a generator that would exhaust a $k = 15$ subspace in a network with 500 scan latches would require an LFSR of 500 stages. Since the LFSR used in this class of generator does not create a maximal length sequence, many different seedings would be needed to exhaust all the $k = 15$ subspaces of the 500 scan latches. An additional problem with such a sequence is that the shift-and-add property does not hold, so a different scheme for distributing such a stimulus across the array of scan latches would be required.

Linear Array Generator

Given a digital network with a parallel scan path structure (such as STUMPS [1]) consisting of h channels each r SRL's deep, we have shown that it takes an LFSR of $r \times h$ stages to implement a two-dimensional window over the structure. In general this is too expensive, so an alternative approach is presented. By constructing a network that spaces the inputs to each of the h channels by r shifts (by utilizing the shift and add property) one can essentially fold the m-sequence into the h channels of the STUMPS structure. In this implementation, we have an LFSR of degree n where $n << r \times h << 2^n - 1$. If one considers the structure as a matrix with r rows and h columns ($r \times h$), one sees that each column contains r elements of the sequence, running from bottom to top. Thus for the ith column, at the bottom in position $\{r, i\}$, bit j of the m-sequence appears. At the top of that column in position $\{1, i\}$, bit $j + r - 1$ appears. At the bottom of the $(i + 1)$th column, bit $j + r$ appears in position $\{r, i + 1\}$. Bit $j + 2r - 1$ appears in $\{1, i + 1\}$ at the top of column $i + 1$. This folding continues until the array is full.

This construction proceeds in an orderly fashion. One first chooses an output that will be used for channel 1. Say this is the output corresponding to x^i. The next channel must be fed by $x^{(i+r)}$, the next by $x^{(i+2r)}$, and so forth. The problem reduces to finding

$$x^k \bmod p(x)$$

where $p(x)$ is the characteristic polynomial of the sequence. A way of performing this calculation is to use the state-space transition matrix approach of Latawiec [13]. The heart of the state-space analysis is the transition matrix T. Given a characteristic polynomial of degree n

$$p(x) = c_n x^n + c_{n-1}x^{n-1} + \cdots + c_2 x^2 + c_1 x + c_0$$

we construct the transition matrix as follows:

$$T = \begin{bmatrix} c_{n-1} & c_{n-2} & \cdots & c_2 & c_1 & c_0 \\ 1 & 0 & \cdots & 0 & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \cdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & 1 & 0 \end{bmatrix}.$$

If we further define the vector v_k as the state of the LFSR at time k (v_k has a "1" in each position that corresponds to a position of the LFSR that has a "1" at time k), we can define the operation

$$\begin{aligned} v_{k+1} &= Tv_k \\ &= T^k v_1 . \end{aligned}$$

We also define that the kth bit u_k of the m-sequence is given by

$$\begin{aligned} u_k &= b_j^t v_k \\ &= b_j^t T^{k-1} v_1 \end{aligned}$$

where

$$\begin{aligned} b_j^t &= 00\cdots1\cdots00 \\ &\quad 12\cdots j\cdots n . \end{aligned}$$

Note that b_j^t is the transpose of b_j. The presence of a "1" in the jth row of an n-vector b means that the output sequence $\{u_k\}$ is taken from the jth register stage. The shifted sequence may be described by

$$u_{k+s}^{(j)} = b_j^t T^{k+s-1} v_1 .$$

On the other hand, we can write

$$u_{k+s}^{(j)} = \sum d_i u_k^{(i)} \bmod 2$$

with "weighting" coefficients d_i, $i = i, \cdots, n$ from $GF(2)$. Combining these last two expressions, we get

$$b_j^t T^s = \sum_{i=1}^{n} d_i b_i^t$$

which, for our purposes, is Latawiec's main result. The vector $\sum d_i b_j^t$ has the simple form of "1's" in places pointed to by indexes i of the nonzero d_i coefficients.

Now go back to our problem of spacing the outputs of an LFSR. Assume that the LFSR contains n stages ($p(x)$ is of degree n). Also, channel 1 will be fed by output i ($i < n$). To calculate the tap points for channel 2, we use the above equation for $b_j^t T^s$ with $s = r$ and $b_j = b_i$. The resulting vector will contain a "1" at each position that must be added (modulo 2) to generate the sequence to feed channel 2 so that it is spaced n shifts down the m-sequence from channel 1. The procedure is repeated for each channel, starting with one channel's vector and applying T r times to get the next channel's tap vector.

Turning now to the previous example, let us construct a linear window generator for the sequence generated by

$$p(x) = x^4 + x + 1$$

over a 6 column by 4 row array. The appropriate transition matrix is

$$T = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

Since the columns have four rows, we need to shift each tap by 4. We let the initial state be defined by $b_1 = [1000]$ and proceed as outlined above. Now b_2, the state shifted 4 times from b_1, is given by

$$\begin{aligned} b_2 &= b_1 T^4 \\ &= [1011] \end{aligned}$$

where

$$T^4 = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}.$$

Likewise,

$$\begin{aligned} b_3 &= b_2 T^4 \\ &= [1110] \\ b_4 &= b_3 T^4 \\ &= [0001] \\ b_5 &= b_4 T^4 \end{aligned}$$

Authorized licensed use limited to: Lakehead University. Downloaded on October 01,2026 at 13:52:26 UTC from IEEE Xplore. Restrictions apply.

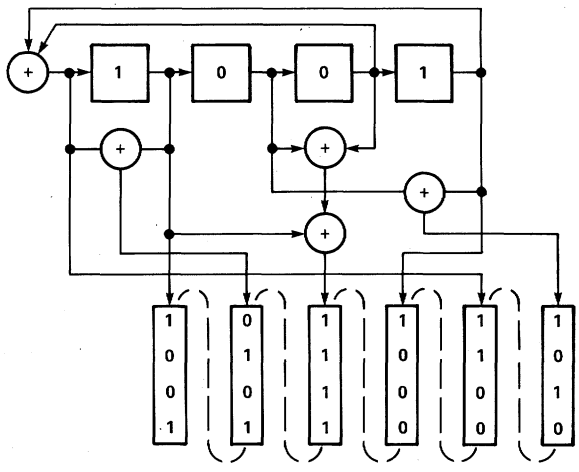


Fig. 9. Compact construction of a linear array.

$$= [0011]$$

$$b_6 = b_5 T^4.$$

$$= [0101]$$

Using the above state vectors to specify the taps to feed the array columns, we have the compact generator shown in Fig. 9.

Having laid the m-sequence out on the STUMPS structure, we now can see that if the channels are thought of as a long contiguous chain of registers, channel 1 top to channel 2 bottom, etc., the m-sequence is intact. Thus all of the properties of an m-sequence apply. The STUMPS structure forms, in effect, a linear window $r \times h$ positions long through which the entire m-sequence is observed in turn. Since all elements of the k-subspace are by definition confined to a portion of the m-sequence at most rh long, the maximum degree that the polynomial representation of the k-subspace may have is rh. We say that the k-subspace may have a span of rh. The occurrence of linear dependencies in m-sequences has been discussed elsewhere [2], [6]. As an example, given a 16th degree m-sequence and a $k = 12$ subspace, it can be shown that there is a probability of 0.06 that a randomly placed k-subspace will contain a linear dependence. Strikingly lower probabilities occur for modest increases in the degree of the m-sequence. For $k = 12$ and $n = 23$, the probability of a linear dependence occurring is 0.00048. Thus, the linear array generator allows the linear dependence analysis to proceed as for a known span of an unmodified m-sequence.

Conclusions

We have shown the construction of pseudorandom array generators that have use in built-in test schemes. The problems of local correlations, both structural and those inherent in the sequence, have been pointed out. A two-dimensional window generator has been shown which insures that all binary patterns occur within the window as the generator cycles. A method of folding the m-sequence on the columns of shift registers so that the array could be treated as an idler register was shown for use where it is undesirable to have a generator of the required length to provide a window.

References

[1] P. H. Bardell and W. H. McAnney, "Self-testing of multi-chip logic modules," in *Proc. 1982 IEEE Test Conf.*, Philadelphia, PA, Nov. 15–18, 1982, pp. 200–204.

[2] P. H. Bardell, "Linear dependencies in m-sequences," IBM Corp., Poughkeepsie, NY, Tech. Rep. 00.3297, Aug. 1984.

[3] Z. Barzilai, D. Coppersmith, and A. Rosenberg, "Exhaustive generation of bit patterns with applications to VLSI self-testing," *IEEE Trans. Comput.*, vol. C-32, pp. 190–194, Feb. 1983.

[4] Z. Barzilai and B. K. Rosen, "Comparison of *AC* self-testing procedures," in *Proc. 1983 Int. Test Conf.*, Philadelphia, PA, Oct. 18–20, 1983, pp. 89–94.

[5] N. Benowitz *et al.*, "An advanced fault isolation system for digital logic," *IEEE Trans. Comput.*, vol. C-24, pp. 489–497, May 1975.

[6] C. L. Chen, private communication, Dec. 1983.

[7] N. G. de Bruijn, "A combinational problem," in *Proc. Nederl. Akad. Wetensch.*, ser. A., vol. 49, pp. 758–764, 1946; also, *Indag. Math.*, vol. 8, pp. 461–467, 1946.

[8] Y. M. El-ziq and H. H. Butt, "A mixed-mode built-in self-test technique using scan path and signature analysis," in *Proc. 1983 Int. Test Conf.*, Philadelphia, PA, Oct. 18–20, 1983, pp. 269–274.

[9] S. W. Golomb, *Shift Register Sequences*. San Francisco, CA: Holden-Day, 1967.

[10] M. Y. Hsiao, "Generating *PN* sequences in parallel," in *Proc. 3rd Ann. Princeton Conf. Inform. Sci. Syst.*, Princeton, NJ, Mar. 1969, pp. 397–401.

[11] W. J. Hurd, "Efficient generation of statistically good pseudonoise by linearly interconnected shift registers," *IEEE Trans. Comput.*, vol. C-23, pp. 146–152, Feb. 1974.

[12] B. Konemann, J. Mucha, and G. Zwiehoff, "Built-in logic block observation techniques," in *Proc. 1979 Int. Test Conf.*, Cherry Hill, NJ, Oct. 1979, pp. 37–41.

[13] K. J. Latawiec, "A method of generation of shifted linear pseudorandom binary sequences," in *Proc. IEE*, vol. 121, pp. 905–906, Aug. 1974.

[14] F. J. MacWilliams and N. J. A. Sloane, "Pseudorandom sequences and arrays," *Proc. IEEE*, vol. 64, Dec. 1976.

[15] T. Nomura, H. Miyakawa, H. Imai, and A. Fukuda, "Some properties of the $\gamma\beta$-plane and its extension to three-dimensional space," *Electron. Commun. in Japan*, vol. 54A, no. 8, pp. 27–34, 1971.

[16] I. S. Reed and R. M. Stewart, "Note on the existence of perfect maps," *IEEE Trans. Inform. Theory*, vol. IT-8, pp. 10–12, 1962.

[17] D. Tang and C. L. Chen, "Logic test pattern generation using linear codes," in *Proc. 13th Fault-Tolerant Comput. Symp., (FTCS-13)*, Milan, Italy, June 1983, pp. 222–226.

[18] S. H. Tsao, "Generation of delayed replicas of maximal-length linear binary sequences," in *Proc. IEE*, vol. 111, pp. 1803–1806, Nov. 1964.

Testable Design of Single-Output Sequential Machines Using Checking Experiments

KEWAL K. SALUJA AND RAMASWAMI DANDAPANI

Abstract **— The problem of testing sequential machines using checking experiments is investigated. A method of modifying sequential machines by adding a controllable input is presented. A procedure is given to construct checking experiments for the modified machine and it is shown that only one output observation is sufficient to determine whether the machine is fault free.**

Index Terms **— Checking experiments, data compaction, design, sequential machines, testable, testing.**

I. Introduction

The explosive growth in the size of digital circuits on a chip has compounded the problem of testing these circuits for correct opera-

Manuscript received December 1, 1984; revised September 16, 1985 and December 24, 1985. This correspondence was presented at the 22nd Annual Allerton Conference on Communications, Control, and Computing, Monticello, IL, October 1984.

K. K. Saluja was with the Department of Electrical and Computer Engineering, University of Newcastle, Newcastle, NSW 2308, Australia. He is now with the Department of Electrical and Computer Engineering, University of Wisconsin, Madison, WI 53706.

R. Dandapani is with the Department of Mathematical and Computer Sciences, Youngstown State University, Youngstown, OH 44555.

IEEE Log Number 8609072.

0018-9340/86/0700-0658$01.00 © 1986 IEEE

Authorized licensed use limited to: Lakehead University. Downloaded on October 01,2026 at 13:52:26 UTC from IEEE Xplore. Restrictions apply.